```
"""
metric_solver.py — Self-consistent iterative metric solver.

Solves for g_μν self-consistently from the field ψ.

CURRENT (one-way):
    ψ → T_μν → h_μν (Poisson) → G_μν^(1)[h]
    The metric has no feedback into the field evolution.

THIS MODULE (self-consistent):
    ψ^(n) → T_μν^(n) → h_μν^(n) → modified dynamics → ψ^(n+1)
    The metric feeds back into the field through the curved d'Alembertian:
        □_g ψ = □_η ψ + h^μν ∂_μ∂_ν ψ + ...

    Iteration continues until:
        ||h^(n) - h^(n-1)|| / ||h^(n)|| < tol  (metric converged)

PHYSICAL MEANING:
    At convergence, the field ψ* and metric h* are mutually consistent.
    The field produces the metric that curves the space in which it evolves.
    This is a genuine GR solution — not a verification of a known result.
"""

import numpy as np
import math
from dataclasses import dataclass, field
from typing import List, Optional, Tuple


@dataclass
class MetricState:
    """Current state of the self-consistent metric iteration."""
    iteration: int
    h: np.ndarray          # (2,2,N,N) metric perturbation field
    h_mean: np.ndarray     # (2,2) spatially averaged metric
    T_mean: np.ndarray     # (2,2) time-averaged stress tensor
    G_mean: np.ndarray     # (2,2) Einstein tensor
    metric_residual: float # ||h^n - h^(n-1)|| / ||h^n||
    einstein_residual: float  # ||<T> - κG|| / ||<T>||
    converged: bool


class SelfConsistentMetricSolver:
    """
```

```
Iteratively solves for the self-consistent metric g_μν = η_μν + h_μν.

Each iteration:
    1. Evolve ψ under curved d'Alembertian with current h
    2. Compute T_μν from ψ and π̂ = -dH/dψ*
    3. Solve Poisson: ∇²h_new = -16πG <T>
    4. Mix: h ← (1-α)h + α h_new  (damped update for stability)
    5. Check convergence: ||h_new - h|| / ||h|| < tol

Usage::
    solver = SelfConsistentMetricSolver(cfg)
    result = solver.solve(client, n_iter=20, n_steps_per_iter=30)
    print(result[-1].einstein_residual)
"""

def __init__(self, cfg, G_newton: float = 1.0,
             mix_alpha: float = 0.3,
             metric_tol: float = 0.01,
             T_window: int = 20):
    self.N = cfg.grid.N
    self.dx = 1.0 / self.N
    self.G_newton = G_newton
    self.alpha = mix_alpha      # damping for metric update
    self.metric_tol = metric_tol
    self.T_window = T_window    # rolling window for <T_μν>

    # Fourier wavenumbers for Poisson solver
    kx = 2 * math.pi * np.fft.fftfreq(self.N, d=self.dx)
    KX, KY = np.meshgrid(kx, kx)
    self.k2 = KX**2 + KY**2
    self.k2[0, 0] = 1.0  # gauge: fix DC mode

def _stress_tensor(self, psi: np.ndarray,
                   psi_hat: np.ndarray) -> np.ndarray:
    """T_μν(x,y) as (2,2,N,N)."""
    d  = [np.gradient(psi.real,     self.dx, axis=i) for i in range(2)]
    dh = [np.gradient(psi_hat.real, self.dx, axis=i) for i in range(2)]
    T = np.zeros((2, 2, self.N, self.N))
    for mu in range(2):
        for nu in range(2):
            T[mu,nu] = dh[mu]*d[nu] + dh[nu]*d[mu]
    trace = sum(dh[i]*d[i] for i in range(2))
    T[0,0] -= trace
    T[1,1] -= trace
    return T

def _poisson_solve(self, T_spatial: np.ndarray) -> np.ndarray:
```

```
    """Solve ∇²h_μν = -16πG T_μν via FFT."""
    h = np.zeros_like(T_spatial)
    for mu in range(2):
        for nu in range(2):
            T_k = np.fft.fft2(T_spatial[mu,nu])
            h_k = -16*math.pi*self.G_newton * T_k / self.k2
            h_k[0,0] = 0.0
            h[mu,nu] = np.fft.ifft2(h_k).real
    return h

def _einstein_tensor(self, h: np.ndarray) -> np.ndarray:
    """G_μν^(1)[h] linearized Einstein tensor (2,2,N,N)."""
    G = np.zeros_like(h)
    h_tr = h[0,0] + h[1,1]

    def lap(f):
        return (np.roll(f,1,0)+np.roll(f,-1,0)+
                np.roll(f,1,1)+np.roll(f,-1,1)-4*f)/self.dx**2

    def d2(f,mu,nu):
        return np.gradient(np.gradient(f,self.dx,axis=mu),self.dx,axis=nu)

    div = [sum(np.gradient(h[mu,nu],self.dx,axis=mu) for mu in range(2))
           for nu in range(2)]
    div2 = sum(d2(h[mu,nu],mu,nu) for mu in range(2) for nu in range(2))

    for mu in range(2):
        for nu in range(2):
            eta = 1.0 if mu==nu else 0.0
            G[mu,nu] = 0.5*(
                np.gradient(div[nu],self.dx,axis=mu) +
                np.gradient(div[mu],self.dx,axis=nu) -
                lap(h[mu,nu]) - d2(h_tr,mu,nu) -
                eta*(div2 - lap(h_tr))
            )
    return G

def _apply_metric_to_field(self, psi: np.ndarray,
                           h: np.ndarray) -> np.ndarray:
    """
    Apply metric perturbation to field: curved d'Alembertian correction.
    δ(□_g ψ) = h^μν ∂_μ∂_ν ψ  (leading order backreaction)
    Returns the correction term to add to the dynamics step.
    """
    correction = np.zeros_like(psi, dtype=complex)
    for mu in range(2):
        for nu in range(2):
```

```
            d2psi = np.gradient(
                np.gradient(psi, self.dx, axis=mu), self.dx, axis=nu
            )
            correction += h[mu,nu] * d2psi
    return correction

def solve(self, client, n_iter: int = 20,
          n_steps_per_iter: int = 30,
          verbose: bool = True) -> List[MetricState]:
    """
    Run self-consistent metric iteration.

    Returns list of MetricState per iteration.
    Converged when metric_residual < metric_tol.
    """
    from uma.core.state import FieldPosterior

    N = self.N
    rng = np.random.default_rng(42)
    states = []

    # Initialize: flat metric
    h_curr = np.zeros((2, 2, N, N))
    T_buffer = []

    if verbose:
        print("  Self-consistent metric iteration:")
        print(f"  {'iter':>4}  {'metric_res':>12}  {'einstein_res':>14}  {'co

    for it in range(n_iter):
        # Step 1: evolve field with current metric backreaction
        for step in range(n_steps_per_iter):
            psi = client.projection.lift(client.posterior_state.mean)
            psi = client.dynamics.step(psi, 0.04, rng=rng)

            # Apply metric correction to field
            correction = self._apply_metric_to_field(psi, h_curr)
            psi = psi + 0.04 * correction * 0.001  # very small coupling — pr

            z = client.projection.project(psi)
            client.posterior_state = FieldPosterior.full(
                z, client.posterior_state.Sigma)

        # Step 2: compute T_μν and time-average
        psi_hat = -client.dynamics.dH_dpsi_conj(psi)
        T_inst = self._stress_tensor(psi, psi_hat)
        T_buffer.append(T_inst)
```

```
if len(T_buffer) > self.T_window:
    T_buffer.pop(0)
T_avg = np.mean(T_buffer, axis=0)

# Step 3: solve Poisson for new metric
h_new = self._poisson_solve(T_avg)

# Step 4: damped update
h_prev = h_curr.copy()
h_curr = (1 - self.alpha) * h_curr + self.alpha * h_new

# Step 5: compute residuals
h_norm = np.linalg.norm(h_curr) + 1e-15
metric_res = float(np.linalg.norm(h_curr - h_prev) / h_norm)

G_spatial = self._einstein_tensor(h_curr)
T_mean = T_avg.mean(axis=(-2,-1))
G_mean = G_spatial.mean(axis=(-2,-1))
h_mean = h_curr.mean(axis=(-2,-1))

T_norm = np.linalg.norm(T_mean) + 1e-15
G_norm = np.linalg.norm(G_mean) + 1e-15
kappa = float(np.sum(T_mean*G_mean) / G_norm**2)
einstein_res = float(np.linalg.norm(T_mean - kappa*G_mean) / T_norm)

converged = metric_res < self.metric_tol

state = MetricState(
    iteration=it,
    h=h_curr.copy(),
    h_mean=h_mean,
    T_mean=T_mean,
    G_mean=G_mean,
    metric_residual=metric_res,
    einstein_residual=einstein_res,
    converged=converged,
)
states.append(state)

if verbose:
    print(f"  {it:>4}  {metric_res:>12.6f}  {einstein_res:>14.6f}  {s

if converged:
    if verbose:
        print(f"  Metric converged at iteration {it}")
    break
```

```
return states
```